

Python Text Summarizer

CS3080 - Shane David

Architecture

- File Reading - file_utils.py:
 - Files need to be read as raw string with normalized whitespace
 - FileUtils class with static methods
- Summarization - summarizer.py:
 - Frequency Based Summarization
 - Transformers Summarization
- User Interface - main.py:
 - Take file path and return summary

Frequency-Based Summarization

- Tokenizes the text into sentences and words using NLTK
- Filters text (stop words and punctuation)
- Counts word frequency and normalizes
- Scores each sentence based on frequency of their words
- Selects top N-highest scoring sentences
- Results is the top sentences from the text with the most frequent words

```
# build a dictionary from words to frequencies ignoring stop words and punctuation
freq = defaultdict(int)
for word in words:
    if word not in stop_words and word not in string.punctuation:
        freq[word] += 1

# normalize the frequencies on a 0-1 scale
max_freq = max(freq.values())
freq = {word: count / max_freq for word, count in freq.items()}
```

```
# score each sentence (sum of its words frequencies)
sentence_scores = defaultdict(float)
for i, sentence in enumerate(sentences):

    # get the words of each sentence
    sentence_words = word_tokenize(sentence.lower())

    # skip empty sentences
    if len(sentence_words) == 0:
        continue

    # increase the score for each word by its frequency
    for word in sentence_words:
        if word in freq:
            sentence_scores[i] += freq[word]

# normalize so that longer sentences are not automatically higher scoring
sentence_scores[i] /= len(sentence_words) ** 0.5
```

You, 2 days ago • added .txt

Transformer Based Summarization

- Uses BART, an encoder-decoder model trained on 300K news article/summary pairs
- Encoder reads the full input and builds an internal representation of its meaning
- Decoder generates a summary word by word
- Results is entirely new sentences based on the content of the recieved text
- Higher computation cost but more human-like and cohesive summaries

```
summarizer_model = None
def get_summarizer_model():

    global summarizer_model

    if summarizer_model is None:

        # import the module
        from transformers import pipeline

        print("Loading summarization model...")

        # create the model
        summarizer_model = pipeline("summarization", model="facebook/bart-large-cnn")

    return summarizer_model
```

```
# the transformers algorithm takes the text, max lenght and min lengths, it creates or fetches
# the model with get_summarizer_model() and then lets that model encode the text and decode it into a summary
def transformer_summarize(text, max_len=130, min_len=30):

    model = get_summarizer_model()
    result = model(text, max_length=max_len, min_length=min_len, do_sample=False, truncation=True)
    return result[0]['summary_text']    You, yesterday • Added transformers algorithm and made basic men...
```

Future Improvements

- Transformers is capped at 1024 tokens
 - Algorithm to separate larger documents into smaller pieces and combine summaries
- Frequency algorithm counts words like “run”, “runs”, “running” as different
 - Algorithm to stem word variants so they count towards the same and the frequency is more accurate
- Expand FileUtils to take in more files